

# Combine GTSeek

John D. Robinson

2026-08-20

## Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Combining datasets for recapture analysis: LGC and GTSeek</b> | <b>1</b>  |
| Compile metadata . . . . .                                       | 2         |
| Extract SNPs . . . . .                                           | 5         |
| Filter LGC data . . . . .                                        | 7         |
| <b>Filter GTSeek data</b>                                        | <b>9</b>  |
| GT-seq Performance . . . . .                                     | 9         |
| Individual filtering . . . . .                                   | 15        |
| <b>Combine LGC and GTSeek data</b>                               | <b>16</b> |
| Individual-level filter (missingness) . . . . .                  | 20        |
| Evaluate Power of the Set . . . . .                              | 25        |
| Test file . . . . .                                              | 28        |

## Combining datasets for recapture analysis: LGC and GTSeek

Some of the samples with DNA yields too low for capture-seq at Rapid Genomics were sent to GTSeek for genotyping-in-thousands by sequencing (GT-seq) genotyping ([Campbell et al. 2015](#)) using a panel of markers developed by Dana Morin at Mississippi State University. We want to combine the two datasets for recapture analyses that follow. To do so, there are several things we need to do here:

1. compile metadata for the samples and add this to our `sampleMetadata` file
2. extract SNPs targeted by the GT-seq panel for hair samples genotyped by capture-seq
3. apply some modest locus-level filters (quality, genotype depth) to LGC data
4. apply some individual-level filters to GTSeek data (based on the individual fuzziness index (IFI) - a measure of potential contamination)
5. combine the two datasets
6. record the number of SNPs genotyped for all samples in our `sampleMetadata` file
7. drop samples that are not sufficiently genotyped prior to recapture analyses
  - for GTSeek samples, at least 100 SNPs genotyped
  - for LGC samples, at least 30% of loci genotyped (in the combined dataset)

We'll walk through these steps below. At the end of the markdown, we include a short section evaluating the power of the marker set via calculation of probabilities of identity for random individuals ( $p_{ID}$ ) and for siblings ( $p_{IDsib}$ ). These measure the chance that two unrelated individuals, or a pair of siblings, would share identical genotypes.

## Compile metadata

First, let's load the data and some files with information about the samples (which were submitted to GTSeek in four batches)...

```
gt <- read.csv("../input/UGA_bbh_Prod1_all_Genotypes.csv", header = TRUE)
gt[1:5,1:8]
```

```
##                               Sample Raw.Reads On.Target.Reads X.On.Target
## 1  GTseek_024_384i5_001_P098.001_5-23      659842           116896      17.72
## 2  GTseek_024_384i5_002_P098.003_928-23      918083           137368      14.96
## 3  GTseek_024_384i5_003_P098.001_65-23      763212             3067       0.40
## 4  GTseek_024_384i5_004_P098.003_929-23      556873           425201      76.36
## 5  GTseek_024_384i5_005_P098.001_82-23      876647             3948       0.45
##    X.GT IFI NW_025576306.1_4455299 NW_025576307.1_2027013
## 1 81.63 0.63                GG                00
## 2 93.20 2.06                GA                CC
## 3 61.22 5.22                GG                00
## 4 76.19 6.42                00                TC
## 5 50.34 8.07                00                TC
```

```
#Create a ugaID column with sample identifiers
gt$ugaID <- sapply(gt$Sample, FUN=function(x) strsplit(x, split="_")[[1]][6])
```

```
#Create an object for metadata
gt.smd <- data.frame(gtsID = gt$Sample,
                     ugaID = gt$ugaID,
                     is.SCR = NA,
                     is.rep = NA,
                     week=NA,
                     year=NA,
                     snare=NA,
                     utmSnare=NA,
                     fqfile=NA,
                     knownSEX=NA,
                     sampSOURCE=NA,
                     region=NA,
                     nSNPgt=NA)
```

```
#To fill in the utm snares, we'll need the translatable.csv file
# this was generated in `1_snareLocations`
translatable <- read.csv("../sampleinfo/translatable.csv", header = TRUE)
```

```
#We also need the source of metadata for these GTSeek samples
#Read sample metadata into R
tmp1 <- read.csv("../input/GTSeek_rd1_Metadata.csv", header = TRUE)
tmp2 <- read.csv("../input/GTSeek_rd2_Metadata.csv", header = TRUE)
tmp3 <- read.csv("../input/GTSeek_rd3_Metadata.csv", header = TRUE)
```

```

tmp4 <- read.csv("../input/GTSeek_rd4_Metadata.csv", header = TRUE)
#Some additional metadata for samples that were originally submitted to LGC
lgcinfo <- read.csv("../input/Finalized Hair Samples w Metadata_Carr.csv",
                    header = TRUE)

colnames(tmp4)[which(colnames(tmp4) == "Site.ID")] <- "Site.Name"
tmp4 <- tmp4[,-which(colnames(tmp4) == "RandomID")]
colnames(tmp4)[which(colnames(tmp4) == "Hair.ID")] <- "RandomID"

keepcols <- c("RandomID", "Site.Name", "Week", "Year")
sampinfo <- rbind(tmp1[,keepcols],
                  tmp2[,keepcols],
                  tmp3[,keepcols],
                  tmp4[,keepcols])

#Clean the dataset
#A few samples had incorrect or missing names, or were duplicated unintentionally
#624-24 was duplicated
gt <- gt[-which(gt$ugaID == "624-24")[1],] #Drop it from gt
gt.smd <- gt.smd[-which(gt.smd$ugaID=="624-24")[1],] # from gt.smd
sampinfo <- sampinfo[-which(sampinfo$RandomID == "624-24")[1],] # and from sampinfo

#A couple of mislabeled samples
gt.smd$ugaID[which(gt.smd$ugaID == "DC-99427AB")] <- "DC-99427B"
gt.smd$ugaID[which(gt.smd$ugaID == "DC-99427Sow")] <- "DC-87805Sow"

#A sample with missing name
gt.smd$ugaID[which(gt.smd$ugaID == "#REF!")] <- "UNK"

#One hair snare without a leading zero in the name
sampinfo$Site.Name[which(sampinfo$Site.Name == "HS 44")] <- "HS 044"

#Going through the loop to fill in metadata...
for(i in 1:nrow(gt.smd)) {
  #gtsID & ugaID are already filled in
  if(gt.smd$ugaID[i] %in% lgcinfo$RandomID) {
    is.LGC <- TRUE
  } else {
    is.LGC <- FALSE
  }

  #Account for NTC samples
  if(length(grep("NTC",gt.smd$ugaID[i])) == 1) {
    next
  }

  #Fill in is.rep
  if(length(grep("R$", gt.smd$ugaID[i])) == 1) {
    gt.smd$is.rep[i] <- ifelse(
      length(which(gt.smd$ugaID == sub("R$", "",gt.smd$ugaID[i]))) == 0,
      FALSE, TRUE)
    tmp.ID <- sub("R$", "",gt.smd$ugaID[i])
  }
}

```

```

} else {
  gt.smd$is.rep[i] <- ifelse(
    length(which(gt.smd$ugaID == paste0(gt.smd$ugaID[i], "R"))) == 0,
    FALSE, TRUE)
  tmp.ID <- gt.smd$ugaID[i]
}

#Account for a few samples that are not in sampinfo
if(gt.smd$is.rep[i] == FALSE &
  length(which(sampinfo$RandomID == tmp.ID)) == 0) {
  gt.smd$is.SCR[i] <- FALSE
  next
}

gt.smd$week[i] <- sampinfo$Week[sampinfo$RandomID == tmp.ID]
gt.smd$year[i] <- sampinfo$Year[sampinfo$RandomID == tmp.ID]

if(length(grep("HS", sampinfo$Site.Name[sampinfo$RandomID == tmp.ID])) > 0) {
  gt.smd$snare[i] <- sampinfo$Site.Name[sampinfo$RandomID == tmp.ID]
  if(gt.smd$year[i] == 2023) {
    gt.smd$utmSnare[i] <- translatable$UTMname[which(translatable$name23 == gt.smd$snare[i])]
  } else if(gt.smd$year[i] == 2024) {
    gt.smd$utmSnare[i] <- translatable$UTMname[which(translatable$name24 == gt.smd$snare[i])]
  }
  gt.smd$region[i] <- "CGA"
  gt.smd$sampSOURCE[i] <- "Hair Sample"
  gt.smd$is.SCR[i] <- TRUE
} else {
  gt.smd$snare[i] <- NA
  gt.smd$utmSnare[i] <- NA
  gt.smd$is.SCR[i] <- FALSE
  gt.smd$sampSOURCE[i] <- sampinfo$Site.Name[sampinfo$RandomID == tmp.ID]
  if(gt.smd$year[i] == 2012) {
    gt.smd$region[i] <- "CGA-H"
  } else {
    gt.smd$region[i] <- "CGA"
  }
}

#nSNPgt is NA for now...
gt.smd$nSNPgt[i] <- NA
#Some of the sexes are probably known, but not recorded in our metadata...
gt.smd$knownSEX[i] <- NA

#If sample was submitted to LGC, pull additional metadata
if(is.LGC) {
  which.row <- which(lgcinfo$Hair.ID == tmp.ID)
  gt.smd$sampSOURCE[i] <- lgcinfo$What[which.row]
  gt.smd$knownSEX[i] <- lgcinfo$Sex[which.row]
  gt.smd$region[i] <- lgcinfo$Region[which.row]
  rm(which.row)
}
rm(is.LGC, tmp.ID)

```

```
}
gt.smd[1:10,]
```

```
##              gtsID   ugaID is.SCR is.rep week year  snare
## 1   GTseek_024_384i5_001_P098.001_5-23   5-23   TRUE  FALSE    6 2023 HS 012
## 2   GTseek_024_384i5_002_P098.003_928-23 928-23   TRUE  FALSE    4 2023 HS 071
## 3   GTseek_024_384i5_003_P098.001_65-23   65-23   TRUE  FALSE    7 2023 HS 009
## 4   GTseek_024_384i5_004_P098.003_929-23 929-23   TRUE  FALSE    5 2023 HS 025
## 5   GTseek_024_384i5_005_P098.001_82-23   82-23   TRUE  FALSE    5 2023 HS 005
## 6   GTseek_024_384i5_006_P098.003_930-23 930-23   TRUE   TRUE    7 2023 HS 028
## 7   GTseek_024_384i5_007_P098.001_174-23 174-23   TRUE  FALSE    3 2023 HS 072
## 8   GTseek_024_384i5_008_P098.003_931-23 931-23   TRUE  FALSE    7 2023 HS 032
## 9   GTseek_024_384i5_009_P098.001_258-23 258-23   TRUE  FALSE    3 2023 HS 072
## 10  GTseek_024_384i5_010_P098.003_932-23 932-23   TRUE  FALSE    4 2023 HS 116
##          utmSnare fqfile knownSEX  sampSOURCE region nSNPgt
## 1  258412x3586648    NA    <NA> Hair Sample    CGA    NA
## 2  266900x3593042    NA    <NA> Hair Sample    CGA    NA
## 3  259209x3585507    NA    <NA> Hair Sample    CGA    NA
## 4  265276x3591787    NA    <NA> Hair Sample    CGA    NA
## 5  252875x3588670    NA    <NA> Hair Sample    CGA    NA
## 6  258399x3594245    NA    <NA> Hair Sample    CGA    NA
## 7  266197x3594132    NA    <NA> Hair Sample    CGA    NA
## 8  263672x3593292    NA    <NA> Hair Sample    CGA    NA
## 9  266197x3594132    NA    <NA> Hair Sample    CGA    NA
## 10 269531x3605091    NA    <NA> Hair Sample    CGA    NA
```

## Extract SNPs

Now we'll use the column names of the `gt` object to create a list of SNPs to extract from our raw vcf file (from LGC) using the `--positions` argument to `vcftools`. In other words, add the loci genotyped by GTSeek to the set of markers that we're using for individual identification. Note that some of these SNPs will not have a minor allele frequency below the filter applied in `3_filterSCR`. We will retain them nonetheless to avoid too much missing data for samples processed via GTSeek.

```
#Column names, avoiding info columns, sexing loci columns, and the ugaID column
gtsSNPs <- colnames(gt)[-c(1:6,grep("Uam",colnames(gt)),ncol(gt))]
length(gtsSNPs)
```

```
## [1] 141
```

```
CPOS <- data.frame(CHROM=rep(NA, length(gtsSNPs)), POS=rep(NA, length(gtsSNPs)))

#column names are CHROM_POS - CHROMs are NW_#####.1
# e.g., NW_025576306.1_4455299
CPOS$CHROM <- sapply(gtsSNPs, FUN=function(x)
  paste(strsplit(x, split="_")[[1]][1:2], collapse="_"))
CPOS$POS <- sapply(gtsSNPs, FUN=function(x)
  strsplit(x, split="_")[[1]][3])

head(CPOS)
```

```
##          CHROM      POS
## 1 NW_025576306.1 4455299
## 2 NW_025576307.1 2027013
## 3 NW_025576331.1 1584277
## 4 NW_025576331.1 441039
## 5 NW_025576393.1 1735059
## 6 NW_025576453.1 20968993
```

```
#Check for loci that are ALREADY in LGC_SCR.vcf
filtSNPs <- data.frame(CHROM=system("grep -v ^# LGC_SCR.vcf | cut -f 1", intern=TRUE),
                      POS = system("grep -v ^# LGC_SCR.vcf | cut -f 2", intern=TRUE))

#Avoid including these loci!!
present <- c()
for(i in 1:nrow(CPOS)) {
  tmp.ind <- which(filtSNPs$CHROM == CPOS$CHROM[i] &
                  filtSNPs$POS == CPOS$POS[i])
  if(length(tmp.ind) > 0) {
    present <- c(present, i)
  }
}
CPOS <- CPOS[~present,]
length(present)
```

```
## [1] 46
```

```
write.table(CPOS, file = "GTSeek_SNPs.txt",
           quote = FALSE, row.names = FALSE, col.names = FALSE)
```

Now extract data for these markers using vcftools...

```
vcftools --vcf UGA_173101_SNPs_Raw.vcf --out GTSeek_SNPs \
         --positions GTSeek_SNPs.txt --recode

#How many are in the output?
grep -v ^# GTSeek_SNPs.recode.vcf | wc -l
```

VCFTools - 0.1.17  
(C) Adam Auton and Anthony Marcketta 2009

Parameters as interpreted:

```
--vcf UGA_173101_SNPs_Raw.vcf
--out GTSeek_SNPs
--positions GTSeek_SNPs.txt
--recode
```

```
Warning: Expected at least 2 parts in INFO entry: ID=AF,Number=A,Type=Float,Description="Estimated alle
Warning: Expected at least 2 parts in INFO entry: ID=PRO,Number=1,Type=Float,Description="Reference alle
Warning: Expected at least 2 parts in INFO entry: ID=PAO,Number=A,Type=Float,Description="Alternate alle
Warning: Expected at least 2 parts in INFO entry: ID=SRP,Number=1,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=SAP,Number=A,Type=Float,Description="Strand balance
```

```

Warning: Expected at least 2 parts in INFO entry: ID=AB,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=ABP,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=RPP,Number=A,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=RPPR,Number=1,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=EPP,Number=A,Type=Float,Description="End Placemen
Warning: Expected at least 2 parts in INFO entry: ID=EPPR,Number=1,Type=Float,Description="End Placemen
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=CIGAR,Number=A,Type=String,Description="The extend
Warning: Expected at least 2 parts in FORMAT entry: ID=GQ,Number=1,Type=Float,Description="Genotype Qua
Warning: Expected at least 2 parts in FORMAT entry: ID=GL,Number=G,Type=Float,Description="Genotype Lik
After filtering, kept 827 out of 827 Individuals
Outputting VCF file...
After filtering, kept 87 out of a possible 319680 Sites
Run Time = 8.00 seconds
87

```

## Filter LGC data

Now apply some modest filters (quality and genotype depth only) to this small dataset to avoid including bad genotypes or markers...

```

#First min quality
vcftools --vcf GTSeek_SNPs.recode.vcf --out LGC_GTS1 \
    --minQ 30 --recode

#Then min genotype depth
vcftools --vcf LGC_GTS1.recode.vcf --out LGC_GTS2 \
    --minDP 10 --recode

#Remove any half-calls that are included in this file
grep ^# LGC_GTS2.recode.vcf > preLGC_GTS.vcf
grep -v ^# LGC_GTS2.recode.vcf | sed -e 's/\.\./[0,1]/\.\.\./g' | sed -e 's/[0,1]\.\.\./\.\.\./g' >> preLGC_GTS.vcf

#Then convert to genotype only format with plink2
plink2 --vcf preLGC_GTS.vcf --out LGC_GTS --recode vcf-4.2 --silent

#Combine these SNPs with those from your previous filtering effort
# using 'bcftools concat' - need to sort and index the file first
bcftools sort LGC_SCR.vcf -O b -o LGC_SCR.bcf
bcftools sort LGC_GTS.vcf -O b -o LGC_GTS.bcf
bcftools index LGC_SCR.bcf
bcftools index LGC_GTS.bcf
bcftools concat -a LGC_SCR.bcf LGC_GTS.bcf -O v -o LGC_allSCRsnps.vcf

#Cleanup
rm *LGC_GTS* GTSeek_SNPs.recode.vcf

```

VCFTools - 0.1.17

(C) Adam Auton and Anthony Marcketta 2009

Parameters as interpreted:

```
--vcf GTSeek_SNPs.recode.vcf
--minQ 30
--out LGC_GTS1
--recode
```

```
Warning: Expected at least 2 parts in INFO entry: ID=AF,Number=A,Type=Float,Description="Estimated alle
Warning: Expected at least 2 parts in INFO entry: ID=PRO,Number=1,Type=Float,Description="Reference alle
Warning: Expected at least 2 parts in INFO entry: ID=PAO,Number=A,Type=Float,Description="Alternate alle
Warning: Expected at least 2 parts in INFO entry: ID=SRP,Number=1,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=SAP,Number=A,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=AB,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=ABP,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=RPP,Number=A,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=RPPR,Number=1,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=EPP,Number=A,Type=Float,Description="End Placement
Warning: Expected at least 2 parts in INFO entry: ID=EPPR,Number=1,Type=Float,Description="End Placemen
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=CIGAR,Number=A,Type=String,Description="The extend
Warning: Expected at least 2 parts in FORMAT entry: ID=GQ,Number=1,Type=Float,Description="Genotype Qua
Warning: Expected at least 2 parts in FORMAT entry: ID=GL,Number=G,Type=Float,Description="Genotype Lik
After filtering, kept 827 out of 827 Individuals
Outputting VCF file...
After filtering, kept 87 out of a possible 87 Sites
Run Time = 0.00 seconds
```

VCfTools - 0.1.17

(C) Adam Auton and Anthony Marcketta 2009

Parameters as interpreted:

```
--vcf LGC_GTS1.recode.vcf
--minDP 10
--out LGC_GTS2
--recode
```

```
Warning: Expected at least 2 parts in INFO entry: ID=AF,Number=A,Type=Float,Description="Estimated alle
Warning: Expected at least 2 parts in INFO entry: ID=PRO,Number=1,Type=Float,Description="Reference alle
Warning: Expected at least 2 parts in INFO entry: ID=PAO,Number=A,Type=Float,Description="Alternate alle
Warning: Expected at least 2 parts in INFO entry: ID=SRP,Number=1,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=SAP,Number=A,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=AB,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=ABP,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=RPP,Number=A,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=RPPR,Number=1,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=EPP,Number=A,Type=Float,Description="End Placement
Warning: Expected at least 2 parts in INFO entry: ID=EPPR,Number=1,Type=Float,Description="End Placemen
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
```



```

Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=CIGAR,Number=A,Type=String,Description="The extend
Warning: Expected at least 2 parts in FORMAT entry: ID=GQ,Number=1,Type=Float,Description="Genotype Qua
Warning: Expected at least 2 parts in FORMAT entry: ID=GL,Number=G,Type=Float,Description="Genotype Lik
After filtering, kept 827 out of 827 Individuals
Outputting VCF file...
After filtering, kept 87 out of a possible 87 Sites
Run Time = 0.00 seconds
Writing to /tmp/bcftools.suzgx4
Merging 1 temporary files
Done
Cleaning
Writing to /tmp/bcftools.TC7xoL
Merging 1 temporary files
Done
Cleaning
Checking the headers and starting positions of 2 files

```

## Filter GTSeek data

The GTSeek dataset has some samples that are poorly genotyped and some that may be contaminated. With that in mind, we're going to drop some samples before combining them with the rest of the data. Before we get to those filters, let's look at the data a little more closely.

## GT-seq Performance

First just the total number of reads per sample in the dataset...

```

#Total Reads
quantile(gt$Raw.Reads) #There are some outliers

```

```

##           0%          25%          50%          75%          100%
##      539.0  531872.5  614148.5  776256.2  1974039.0

```

```

mean(gt$Raw.Reads)

```

```

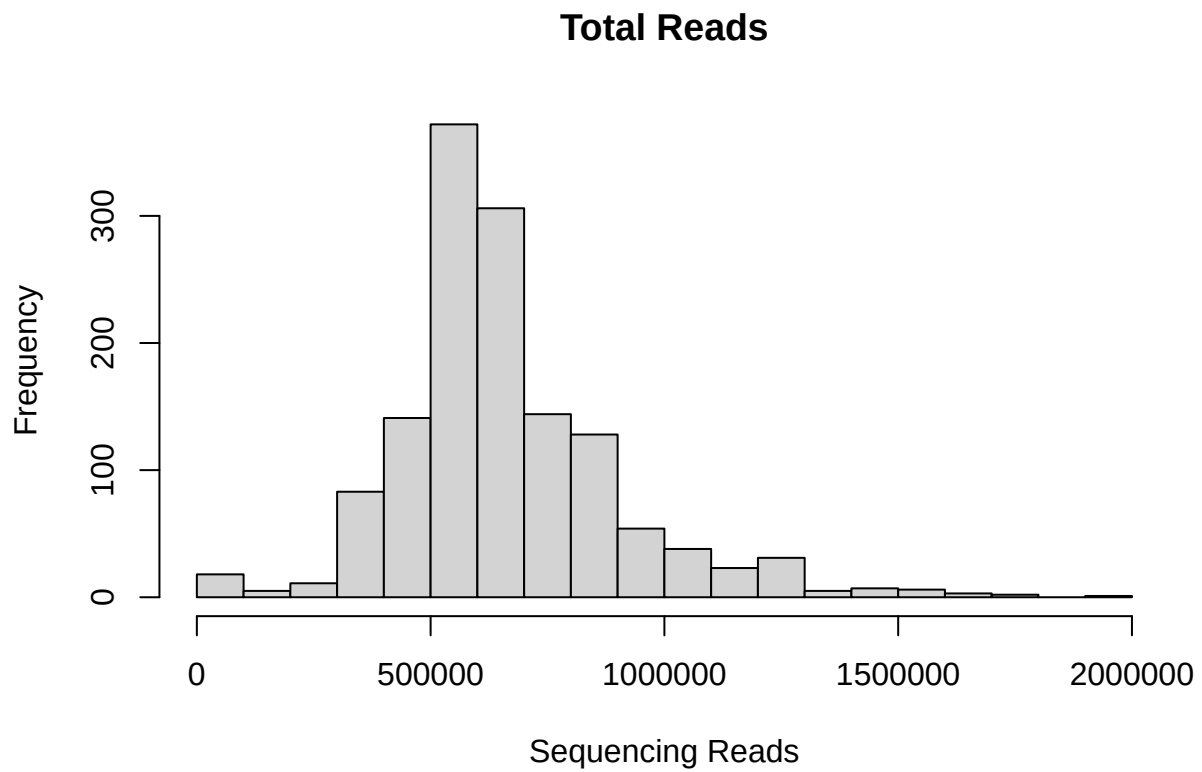
## [1] 660728.8

```

```

hist(gt$Raw.Reads, breaks = 20,
     main = "Total Reads", xlab = "Sequencing Reads")

```



Now the percentage of loci genotyped per sample...

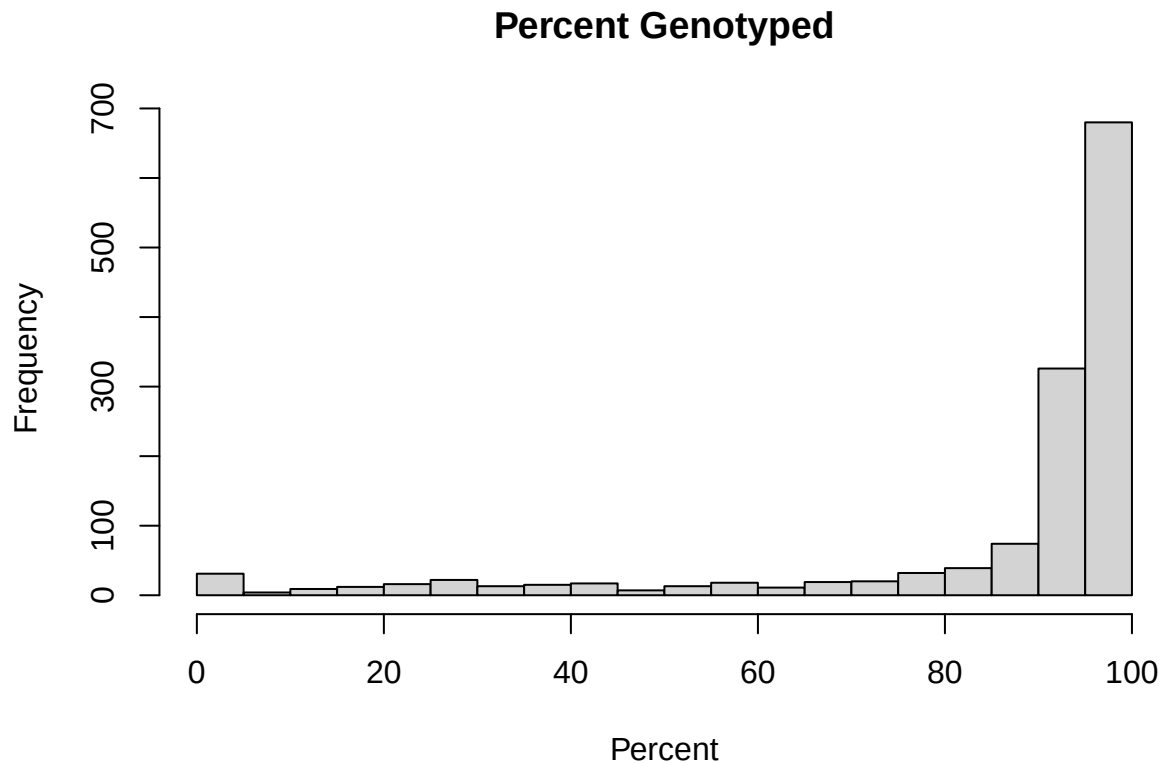
```
#Percent genotyped  
quantile(gt$X.GT)
```

```
##      0%   25%   50%   75%  100%  
##  0.00 88.44 94.56 95.92 98.64
```

```
mean(gt$X.GT)
```

```
## [1] 84.5604
```

```
hist(gt$X.GT, breaks = 20,  
      main = "Percent Genotyped", xlab = "Percent")
```



```
length(which(gt$X.GT > 90))/nrow(gt) #What proportion are genotyped at >90%
```

```
## [1] 0.7300435
```

Finally, the *individual fuzziness index* (IFI). This index is described briefly in the header for some of the functions in Nate Campbell's [GTseek\\_utils](#) GitHub repository (the “[GTseq\\_Genotyper](#)” function, for instance). In the header, IFI is described as:

*“a measure of DNA cross contamination... calculated using read counts from background signal at homozygous and No-Call loci. Low scores are better than high scores.”*

```
#IFI
quantile(gt$IFI)
```

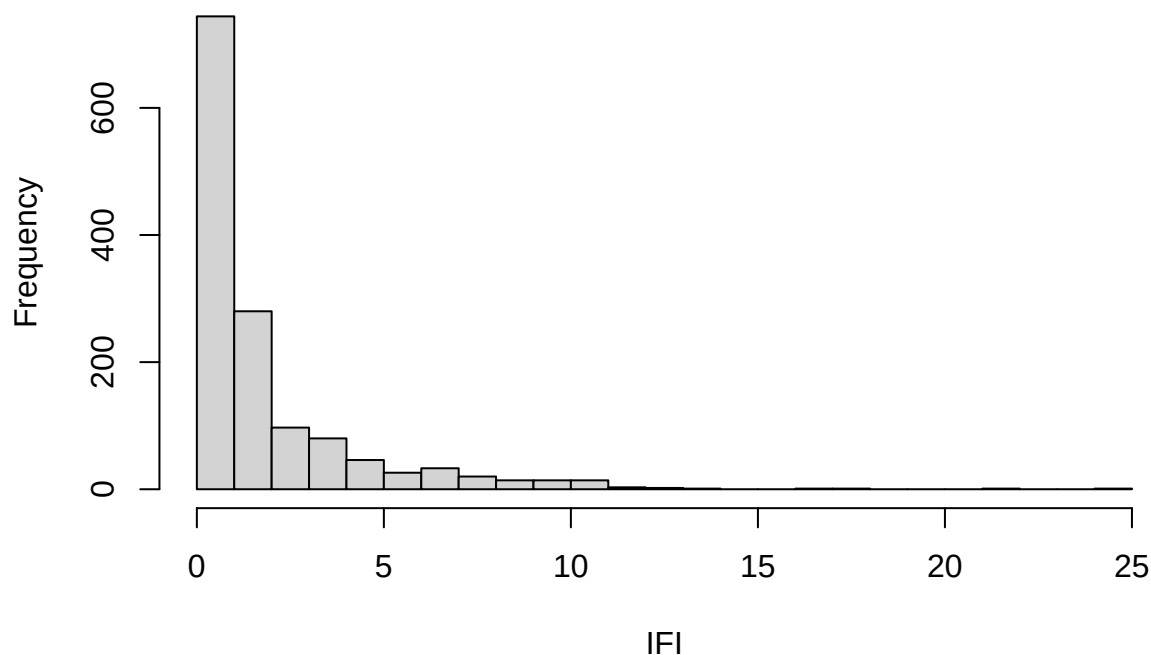
```
##      0%      25%      50%      75%     100%
## 0.00  0.44  0.93  2.07 25.00
```

```
mean(gt$IFI)
```

```
## [1] 1.848534
```

```
hist(gt$IFI, breaks = 20,
     main = "Individual Fuzziness Index", xlab = "IFI")
```

## Individual Fuzziness Index



Does IFI tell us anything about error rates? To assess this, we can look at our technical replicates...

```
#Find tech. reps
repID <- c(gt$ugaID[grepl("-23R", gt$ugaID)],
          gt$ugaID[grepl("-24R", gt$ugaID)])
ogID <- gsub("R","",repID)
comp.rep <- data.frame(ogID, repID,
                      og.XGT=NA, rep.XGT=NA,
                      ogIFI=NA, repIFI=NA,
                      overlap=NA, mismatch=NA, error=NA)
for(i in 1:nrow(comp.rep)) {
  #Check that both samples are present
  tmp.len <- length(which(gt$ugaID %in% c(repID[i], ogID[i])))
  if(tmp.len != 2) {
    stop(paste("Ooops! There are",tmp.len,"samples for",ogID[i]))
  }
  comp.rep$og.XGT[i] <- gt$X.GT[gt$ugaID == comp.rep$ogID[i]]
  comp.rep$rep.XGT[i] <- gt$X.GT[gt$ugaID == comp.rep$repID[i]]
  comp.rep$ogIFI[i] <- gt$IFI[gt$ugaID == comp.rep$ogID[i]]
  comp.rep$repIFI[i] <- gt$IFI[gt$ugaID == comp.rep$repID[i]]

  tmp.gt <- gt[which(gt$ugaID %in% c(comp.rep$ogID[i],
                                   comp.rep$repID[i])),
               -c(1:6,ncol(gt)))]
  #Keep only overlapping calls
  tmp.gt <- tmp.gt[,which(apply(tmp.gt, 2,
```

```

FUN=function(x)
  sum(x %in% c("0", "00"))==0)]
comp.rep$overlap[i] <- ncol(tmp.gt)
comp.rep$mismatch[i] <- sum(tmp.gt[1,] != tmp.gt[2,])
comp.rep$error[i] <- comp.rep$mismatch[i] / comp.rep$overlap[i]
rm(tmp.gt, tmp.len)
}

comp.rep

```

| ##    | ogID    | repID    | og.XGT | rep.XGT | ogIFI | repIFI | overlap | mismatch | error       |
|-------|---------|----------|--------|---------|-------|--------|---------|----------|-------------|
| ## 1  | 930-23  | 930-23R  | 95.92  | 96.60   | 1.12  | 1.30   | 141     | 0        | 0.000000000 |
| ## 2  | 781-23  | 781-23R  | 95.92  | 95.92   | 0.67  | 0.78   | 140     | 0        | 0.000000000 |
| ## 3  | 651-23  | 651-23R  | 94.56  | 95.92   | 0.47  | 0.63   | 139     | 0        | 0.000000000 |
| ## 4  | 798-23  | 798-23R  | 95.92  | 96.60   | 0.30  | 0.42   | 141     | 0        | 0.000000000 |
| ## 5  | 716-23  | 716-23R  | 83.67  | 87.76   | 4.68  | 3.60   | 112     | 7        | 0.062500000 |
| ## 6  | 935-23  | 935-23R  | 95.92  | 96.60   | 0.30  | 0.33   | 140     | 0        | 0.000000000 |
| ## 7  | 424-23  | 424-23R  | 69.39  | 36.05   | 5.33  | 3.33   | 39      | 5        | 0.128205128 |
| ## 8  | 466-23  | 466-23R  | 19.05  | 19.05   | 4.77  | 3.04   | 7       | 0        | 0.000000000 |
| ## 9  | 286-23  | 286-23R  | 3.40   | 2.04    | 0.00  | 2.56   | 2       | 0        | 0.000000000 |
| ## 10 | 311-23  | 311-23R  | 52.38  | 30.61   | 1.96  | 2.14   | 38      | 3        | 0.078947368 |
| ## 11 | 564-24  | 564-24R  | 94.56  | 95.24   | 0.85  | 0.85   | 139     | 0        | 0.000000000 |
| ## 12 | 517-24  | 517-24R  | 51.70  | 34.01   | 6.79  | 9.04   | 33      | 3        | 0.090909091 |
| ## 13 | 726-24  | 726-24R  | 84.35  | 75.51   | 4.20  | 7.03   | 98      | 13       | 0.132653061 |
| ## 14 | 980-24  | 980-24R  | 97.96  | 97.28   | 0.22  | 0.29   | 143     | 0        | 0.000000000 |
| ## 15 | 1306-24 | 1306-24R | 96.60  | 96.60   | 0.25  | 0.29   | 141     | 0        | 0.000000000 |
| ## 16 | 567-24  | 567-24R  | 94.56  | 95.92   | 0.76  | 0.78   | 139     | 0        | 0.000000000 |
| ## 17 | 881-24  | 881-24R  | 95.92  | 92.52   | 0.46  | 1.79   | 134     | 0        | 0.000000000 |
| ## 18 | 2972-24 | 2972-24R | 95.92  | 96.60   | 0.58  | 1.27   | 139     | 0        | 0.000000000 |
| ## 19 | 1384-24 | 1384-24R | 73.47  | 66.67   | 9.94  | 13.05  | 78      | 29       | 0.371794872 |
| ## 20 | 933-24  | 933-24R  | 94.56  | 95.24   | 1.08  | 0.77   | 137     | 0        | 0.000000000 |
| ## 21 | 1522-24 | 1522-24R | 96.60  | 95.92   | 0.96  | 0.93   | 141     | 1        | 0.007092199 |
| ## 22 | 3058-24 | 3058-24R | 96.60  | 96.60   | 0.26  | 0.29   | 141     | 0        | 0.000000000 |
| ## 23 | 2665-24 | 2665-24R | 94.56  | 95.24   | 0.87  | 0.84   | 138     | 0        | 0.000000000 |
| ## 24 | 2869-24 | 2869-24R | 97.28  | 96.60   | 0.37  | 0.49   | 142     | 0        | 0.000000000 |
| ## 25 | 2926-24 | 2926-24R | 91.84  | 87.76   | 2.32  | 3.47   | 122     | 0        | 0.000000000 |
| ## 26 | 151-24  | 151-24R  | 95.92  | 93.20   | 0.35  | 0.40   | 136     | 0        | 0.000000000 |
| ## 27 | 396-24  | 396-24R  | 89.12  | 84.35   | 1.63  | 2.68   | 118     | 1        | 0.008474576 |
| ## 28 | 191-24  | 191-24R  | 24.49  | 22.45   | 2.81  | 5.10   | 16      | 4        | 0.250000000 |
| ## 29 | 500-24  | 500-24R  | 95.92  | 96.60   | 0.17  | 0.19   | 141     | 0        | 0.000000000 |

```
cor.test(comp.rep$repIFI, comp.rep$ogIFI)
```

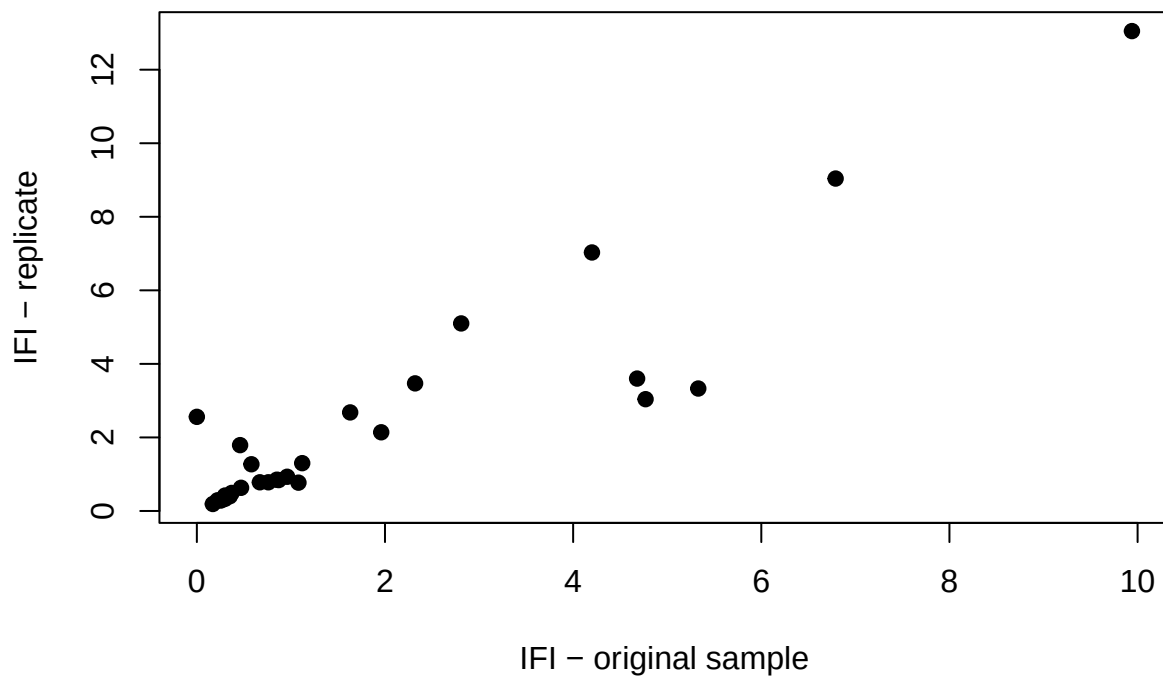
```

##
## Pearson's product-moment correlation
##
## data: comp.rep$repIFI and comp.rep$ogIFI
## t = 12.004, df = 27, p-value = 2.464e-12
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## 0.8305545 0.9609892
## sample estimates:

```

```
##      cor
## 0.9177074
```

```
plot(comp.rep$repIFI ~ comp.rep$logIFI,
      xlab = "IFI - original sample",
      ylab = "IFI - replicate",
      pch = 19)
```

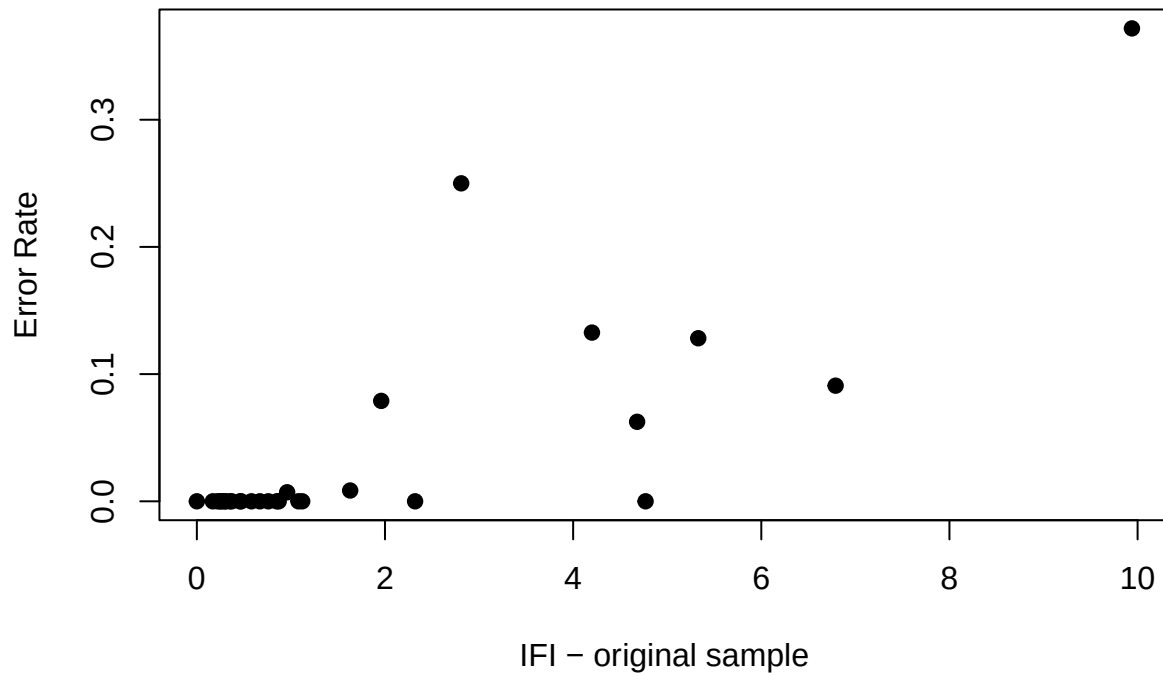


```
cor.test(comp.rep$logIFI, comp.rep$error)
```

```
##
## Pearson's product-moment correlation
##
## data: comp.rep$logIFI and comp.rep$error
## t = 6.7925, df = 27, p-value = 2.7e-07
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  0.6033886 0.8990458
## sample estimates:
##      cor
## 0.7942497
```

```
plot(comp.rep$error ~ comp.rep$logIFI,
      xlab = "IFI - original sample",
```

```
ylab = "Error Rate",
pch=19)
```



So the IFI is very similar between the two technical replicates, and it does seem to be predictive of mismatches between the replicates. This should be useful for identifying poorly genotyped samples that should be removed from the dataset.

## Individual filtering

It seems reasonable to remove samples for either of two reasons, i) low rate of genotyping or ii) high IFI. In their paper describing GT-seq, [Campbell et al. \(2015\)](#) considered samples that were genotyped at less than 50% of SNPs as failed samples. We'll apply a more stringent filter for retaining samples below. For now, drop samples due to high IFI only...

```
#Identify samples with high IFI
drop.me <- which(gt$IFI > 2)
length(drop.me)
```

```
## [1] 354
```

```
#Now remove from genotype and metadata
nrow(gt)
```

```
## [1] 1378
```

```
gt <- gt[-drop.me,]
nrow(gt)
```

```
## [1] 1024
```

## Combine LGC and GTSeek data

Now convert genotypes to VCF format (0/0, 0/1, 1/1), taking care to assign REF alleles to 0 and ALT alleles to 1, to match the conventions in the VCF. Then combine with LGC\_SCR.vcf and check that PLINK, vcftools, and vcfR all recognize the combined output file as a vcf. If this works, you should be able to calculate the pairwise kinship coefficients in PLINK2 and perform the clone analysis in COLONY to identify recaptures across the combined dataset.

```
#First create a file with CHROM, POS, REF, and ALT for all SNPs in LGC_SCR
# This will be used to convert the CC, CT, TT genotypes to 0/0, 0/1, 1/1
# with appropriate coding for ref and alt alleles (0 an 1, respectively)
system("echo 'CHROM\tPOS\tREF\tALT' > scrSNPs.txt")
system("cat scrSNPs.txt", intern = TRUE)
```

```
## [1] "CHROM\tPOS\tREF\tALT"
```

```
#SNP rows of the vcf | CHROM, POS, REF, and ALT columns
system("grep -v ^# LGC_allSCRsnps.vcf | cut -f 1-2,4-5 >> scrSNPs.txt")
system("wc -l scrSNPs.txt")
system("head scrSNPs.txt")
```

```
#Read this file with CHROM, POS, REF, and ALT for all SNPs into R
alleleState <- read.table("scrSNPs.txt", header = TRUE)
head(alleleState)
```

```
##           CHROM      POS REF ALT
## 1 NW_025576306.1 1250049   A   G
## 2 NW_025576306.1 1625051   C   A
## 3 NW_025576306.1 4425067   C   T
## 4 NW_025576306.1 4450032   A   G
## 5 NW_025576306.1 4455299   A   G
## 6 NW_025576306.1 5225027   T   C
```

```
#Focus on genotypes at SNPs NOT used for sex determination
geno <- gt[,-c(1:6,grep("Uam", colnames(gt)),ncol(gt))]
dim(geno)
```

```
## [1] 1024 141
```

```
#Make an object to fill in with vcf-formatted genotype data
vcfFormat <- matrix(data = "./.", nrow = nrow(alleleState), ncol = nrow(geno))
for(a in 1:nrow(vcfFormat)) {
  #Column names of the GTSeek data file are CHROM_POS
  tmp.colname <- paste0(alleleState$CHROM[a], "_", alleleState$POS[a])
```



```

tmp.col <- which(colnames(geno) == tmp.colname)
if(length(tmp.col) > 0) { #If this SNP from the LGC_SCR vcf is present in GTSeek data
  #If REF=C and ALT=T
  #The cells where the genotype is CC --> 0/0
  vcfFormat[a,which(geno[,tmp.col] == paste0(alleleState$REF[a],alleleState$REF[a]))] <- "0/0"
  #Cells where genotype is CT or TC --> 0/1
  vcfFormat[a,which(geno[,tmp.col] == paste0(alleleState$REF[a],alleleState$ALT[a]) |
    geno[,tmp.col] == paste0(alleleState$ALT[a],alleleState$REF[a]))] <- "0/1"
  #Cells where genotype is TT --> 1/1
  vcfFormat[a,which(geno[,tmp.col]==paste0(alleleState$ALT[a],alleleState$ALT[a]))] <- "1/1"
}
}
dim(vcfFormat)

```

```
## [1] 1231 1024
```

```

#There will be lots of missing data here
vcfFormat[1:20,1:5]

```

```

##      [,1] [,2] [,3] [,4] [,5]
## [1,] "./" "./" "./" "./" "./"
## [2,] "./" "./" "./" "./" "./"
## [3,] "./" "./" "./" "./" "./"
## [4,] "./" "./" "./" "./" "./"
## [5,] "1/1" "1/1" "0/1" "0/1" "1/1"
## [6,] "./" "./" "./" "./" "./"
## [7,] "./" "./" "./" "./" "./"
## [8,] "./" "./" "./" "./" "./"
## [9,] "./" "./" "./" "./" "./"
## [10,] "./" "0/1" "0/1" "0/1" "1/1"
## [11,] "./" "./" "./" "./" "./"
## [12,] "./" "./" "./" "./" "./"
## [13,] "0/0" "0/1" "0/0" "./" "0/0"
## [14,] "./" "./" "./" "./" "./"
## [15,] "./" "./" "./" "./" "./"
## [16,] "1/1" "0/1" "0/0" "0/1" "0/0"
## [17,] "./" "./" "./" "./" "./"
## [18,] "./" "./" "./" "./" "./"
## [19,] "./" "./" "./" "./" "./"
## [20,] "./" "./" "./" "./" "./"

```

```

#Do these match?
test.row <- which(apply(vcfFormat,1,FUN=function(x) length(unique(x))) > 1)[1]
alleleState[test.row,]

```

```

##      CHROM      POS REF ALT
## 5 NW_025576306.1 4455299  A  G

```

```
gt[1:5,paste0(alleleState$CHROM[test.row],"_",alleleState$POS[test.row])]
```

```
## [1] "GG" "GG" "GA" "GA" "GG"
```

```
vcfFormat[test.row,1:5]
```

```
## [1] "1/1" "1/1" "0/1" "0/1" "1/1"
```

```
#How many SNPs are in the dataset (i.e., not all ./.)  
sum(apply(vcfFormat, 1, FUN=function(x) length(unique(x))) > 1)
```

```
## [1] 132
```

```
#Now write this to a file...  
colnames(vcfFormat) <- paste0("GTS_",gt.smd$ugaID[-drop.me])  
dim(vcfFormat)
```

```
## [1] 1231 1024
```

```
write.table(vcfFormat, sep = "\t", file = "extraCols.vcf",  
            quote = FALSE, row.names = FALSE, col.names = TRUE)
```

Now we append it to the VCF (cbind in R = paste in bash) and save the output. Let's try to do this by first condensing the vcf into a simpler format. Note that we're working with a simple VCF output by PLINK as it is. The "FORMAT" here is only GT (genotypes only, no depth, quality, allele depth, genotype likelihoods, etc.).

```
#Drop all header lines except the column labels  
grep -v ^## LGC_allSCRsnp.vcf > lgc.vcf  
#Paste (cbind) the SNP rows of the LGC_SCR with the extraCols object created above  
paste lgc.vcf extraCols.vcf > out.vcf  
#Now retrieve the header lines  
grep ^## LGC_allSCRsnp.vcf > fullout.vcf  
#And append the combined genotype object to the headers  
cat out.vcf >> fullout.vcf
```

Check to make sure that all loci have exactly two alleles. Some of the markers from the GTSeek panel have 3!

```
vcftools --vcf fullout.vcf --out fullout \  
        --freq
```

VCFTools - 0.1.17

(C) Adam Auton and Anthony Marcketta 2009

Parameters as interpreted:

```
--vcf fullout.vcf  
--freq  
--out fullout
```

```
Warning: Expected at least 2 parts in INFO entry: ID=AF,Number=A,Type=Float,Description="Estimated allele frequency in the sample" (pos=  
Warning: Expected at least 2 parts in INFO entry: ID=PRO,Number=1,Type=Float,Description="Reference allele" (pos=  
Warning: Expected at least 2 parts in INFO entry: ID=PAO,Number=A,Type=Float,Description="Alternate allele" (pos=
```

```

Warning: Expected at least 2 parts in INFO entry: ID=SRP,Number=1,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=SAP,Number=A,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=AB,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=ABP,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=RPP,Number=A,Type=Float,Description="Read Placement
Warning: Expected at least 2 parts in INFO entry: ID=RPPR,Number=1,Type=Float,Description="Read Placement
Warning: Expected at least 2 parts in INFO entry: ID=EPP,Number=A,Type=Float,Description="End Placement
Warning: Expected at least 2 parts in INFO entry: ID=EPPR,Number=1,Type=Float,Description="End Placement
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=CIGAR,Number=A,Type=String,Description="The extend
After filtering, kept 1851 out of 1851 Individuals
Outputting Frequency Statistics...
After filtering, kept 1231 out of a possible 1231 Sites
Run Time = 1.00 seconds

```

Now read this file into R and identify sites that have exactly two alleles in the dataset...

```

snpfreq <- read.table("fullout.frq", skip = 1, fill=TRUE)
colnames(snpfreq)[3] <- "N_ALLELES"
biallelic <- snpfreq[which(snpfreq$N_ALLELES == 2),c(1,2)]

fixed.sites <- c(grep(":1", biallelic$A1),
                 grep(":1", biallelic$A2),
                 grep(":1", biallelic$A3))

fixed.sites

```

```
## integer(0)
```

```
biallelic[fixed.sites,]
```

```
## [1] V1 V2
## <0 rows> (or 0-length row.names)
```

```

if(length(fixed.sites) > 0) {
  biallelic <- biallelic[-fixed.sites,c(1,2)]
}

nrow(biallelic)

```

```
## [1] 1226
```

```

write.table(biallelic, file = "biallelic.txt",
            quote = FALSE, row.names = FALSE, col.names = FALSE)

```

Now keep only this list of SNP loci...

```
vcftools --vcf fullout.vcf --out prefinal \
--positions biallelic.txt \
--remove-filtered-all --recode
```

VCFTools - 0.1.17

(C) Adam Auton and Anthony Marcketta 2009

Parameters as interpreted:

```
--vcf fullout.vcf
--out prefinal
--positions biallelic.txt
--recode
--remove-filtered-all
```

```
Warning: Expected at least 2 parts in INFO entry: ID=AF,Number=A,Type=Float,Description="Estimated alle
Warning: Expected at least 2 parts in INFO entry: ID=PRO,Number=1,Type=Float,Description="Reference alle
Warning: Expected at least 2 parts in INFO entry: ID=PAO,Number=A,Type=Float,Description="Alternate alle
Warning: Expected at least 2 parts in INFO entry: ID=SRP,Number=1,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=SAP,Number=A,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=AB,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=ABP,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=RPP,Number=A,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=RPPR,Number=1,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=EPP,Number=A,Type=Float,Description="End Placement
Warning: Expected at least 2 parts in INFO entry: ID=EPPR,Number=1,Type=Float,Description="End Placemen
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=CIGAR,Number=A,Type=String,Description="The extend
After filtering, kept 1851 out of 1851 Individuals
Outputting VCF file...
After filtering, kept 1226 out of a possible 1231 Sites
Run Time = 0.00 seconds
```

This all worked, so we should be in good shape for downstream analyses with this file!

## Individual-level filter (missingness)

Lastly, we want to check to see how many SNPs are genotyped for each individual in the dataset and combine our metadata files into a version with both LGC and GTSeek samples! We also want to remove samples that are genotyped at fewer than 100 SNPs (GTSeek) or <50% of the SNPs in the combined dataset (LGC).

```
#Get missing data information for each individual
system("vcftools --vcf prefinal.recode.vcf --out prefinal --missing-indv")

#Read this file into R
missSNPs <- read.table("prefinal.imiss", header = TRUE)

#Combine the two metadata files - LGC & GTSeek
```

```

lgc.smd <- read.csv("../sampleinfo/sampleMetadata_nSNPgt.csv", header = TRUE)
colnames(lgc.smd)[1] <- "extID"
colnames(gt.smd)[1] <- "extID"
smd <- rbind(lgc.smd, gt.smd)

smd$vcfID <- c(lgc.smd$extID,
               paste0("GTS_",gt.smd$ugaID))
smd$FILT <- NA

#Update nSNPgt for all samples
for(i in 1:nrow(smd)) {
  tmp <- which(missSNPs$INDV == smd$vcfID[i])
  if(length(tmp) > 0) {
    smd$nSNPgt[i] <- missSNPs$N_DATA[tmp]-missSNPs$N_MISS[tmp]
  }
}

#Set the thresholds
GTS_thresh <- 100
LGC_thresh <- missSNPs$N_DATA[1]*0.3

for(i in 1:nrow(smd)) {
  tmp <- which(missSNPs$INDV == smd$vcfID[i])
  if(length(tmp) > 0) {
    if(length(grep("GTS_",smd$vcfID[i])) > 0) {
      if(smd$nSNPgt[i] < GTS_thresh) {
        smd$FILT[i] <- TRUE
      } else {
        smd$FILT[i] <- FALSE
      }
    } else if(length(grep("UGA_",smd$vcfID[i])) > 0) {
      if(smd$nSNPgt[i] < LGC_thresh) {
        smd$FILT[i] <- TRUE
      } else {
        smd$FILT[i] <- FALSE
      }
    }
  } else {
    smd$FILT[i] <- TRUE
  }
  rm(tmp)
}

#Number removed from dataset
sum(smd$FILT)

```

```
## [1] 526
```

```

#Number NOT removed
sum(!smd$FILT)

```

```
## [1] 1679
```

```
#Number of GTSeek samples with at least 100 SNPs genotyped
sum(smd$nSNPgt[grep("GTS_", smd$vcfID)] >= GTS_thresh, na.rm = TRUE)
```

```
## [1] 993
```

```
#Number of LGC samples with at least 30% of SNPs genotyped
sum(smd$nSNPgt[grep("UGA_", smd$vcfID)] >= LGC_thresh, na.rm = TRUE)
```

```
## [1] 686
```

```
#These should match
sum(!smd$FILT) == sum(smd$nSNPgt[grep("GTS_", smd$vcfID)] >= GTS_thresh, na.rm = TRUE) +
  sum(smd$nSNPgt[grep("UGA_", smd$vcfID)] >= LGC_thresh, na.rm = TRUE)
```

```
## [1] TRUE
```

```
#Number of SNPs genotyped per sample
smd$nSNPgt[which(smd$FILT == FALSE)]
```

```
##      [1]  988 1104  849 1083  743  957  746 1067 1079 1112  993 1185  614 1062
##      [15]  870 1186 1222  603 1094 1223 1191 1198 1051 1141  944  970 1211 1110
##      [29] 1076 1057 1211 1186 1208 1212 1098  709 1049  871  763 1103 1212 1208
##      [43] 1207 1115 1161 1219  890 1034 1177 1217 1154 1077 1134 1087 1214 1207
##      [57] 1199  951 1167 1170 1160 1198 1200 1111  745 1166  919 1199 1118  888
##      [71] 1191 1214 1209 1125 1143 1214 1212 1145 1193  486 1222 1215 1169  770
##      [85] 1145  941 1032 1099 1047 1191 1072 1220 1153 1202 1198 1141 1195  829
##      [99] 1192 1206 1167 1017 1155  451  947  460  434 1192 1151 1025 1032 1143
##     [113]  790  474 1086  857  932 1014  949 1115  888 1044  953 1069  769 1201
##     [127] 1200 1136 1096  852 1211 1050 1173 1164 1199  885 1162  839 1193 1216
##     [141] 1139 1068 1102 1121 1136  709 1185 1188  764 1203 1215 1218 1109 1196
##     [155]  773 1218 1209 1045 1146 1209 1173  926 1082 1183  492 1168 1148 1068
##     [169] 1215 1208 1218 1145 1223 1072  858 1221 1212 1217 1108 1224 1122 1218
##     [183] 1218 1218 1200 1221 1211 1154 1209  868  786 1216 1212  432 1220 1214
##     [197] 1220 1216  490 1200 1126  893 1170 1179  619  838 1208 1159  952 1199
##     [211]  498 1081  974  747 1138 1205 1200 1120 1206  987 1194 1219  859 1072
##     [225] 1214  650  931  773 1223 1222 1222 1217  461 1034 1220  573 1220  808
##     [239]  742 1160 1223 1216  929 1218 1156 1208 1174  727 1158 1193 1033 1220
##     [253] 1203 1219 1219 1195  868  544 1205 1186  513 1136  822 1217  929 1207
##     [267]  588  528  682 1207 1202 1211 1139 1210 1191 1222 1223  603 1186 1203
##     [281] 1219 1174 1214  872 1095  368 1204 1220 1212 1193 1152 1220 1218 1221
##     [295] 1223 1089 1220 1176 1197 1156 1108  374 1155 1201  839 1220 1212 1216
##     [309] 1093  995 1182 1217 1221 1215 1220  892  790 1222 1185 1222 1024 1223
##     [323]  435 1222  990 1211 1020 1223 1193 1216 1219 1222 1222 1216  692 1194
##     [337] 1187 1222 1216  809 1221 1059 1195 1222 1188 1223 1221 1192 1000 1183
##     [351]  876 1168 1198  744 1040 1222 1221 1221 1173 1221 1143 1190 1214 1209
##     [365]  966 1218 1221 1207 1222 1188 1194 1085 1144 1001 1091  991 1159 1116
##     [379] 1189 1195 1087 1207 1207  635 1164 1212 1165  636 1016 1209 1131 1136
##     [393] 1186 1046 1107 1123  593 1169 1187  621 1209 1220 1207 1136 1209 1070
##     [407] 1209 1218 1203 1217 1219 1204 1206 1199 1218 1215 1221 1095  910 1065
##     [421] 1136 1217  856 1191  539 1214 1218 1216 1166  384 1205 1214 1209 1085
##     [435] 1214 1083 1175 1188 1172 1129 1128 1214 1216 1216  934 1134 1213 1200
```

```

## [449] 1218 482 1106 747 1154 1181 1200 1136 1208 665 1202 1207 1195 1193
## [463] 1204 1218 1137 1218 1215 1218 1214 814 1213 965 1218 647 994 1169
## [477] 1207 1212 1198 1020 988 1167 645 765 1213 1033 1219 1186 919 1167
## [491] 1167 629 1182 992 1214 1212 1056 705 1151 904 745 1069 1184 1184
## [505] 763 1217 522 1206 402 1213 374 1105 1055 1213 942 1075 1166 1137
## [519] 1157 596 907 1177 1005 1214 1152 422 1066 534 1209 714 1209 1195
## [533] 544 1198 878 1190 1135 1039 428 1127 1213 1213 982 715 432 1200
## [547] 1156 1212 852 1160 1207 912 1092 551 584 563 1038 1192 1030 1096
## [561] 1173 1154 815 1085 1063 888 990 704 1200 887 850 1203 1217 1178
## [575] 446 1197 980 1214 827 1212 1217 1201 1086 1215 1201 1194 948 505
## [589] 1167 1110 1131 1176 818 1088 1208 432 1086 499 748 1167 782 1216
## [603] 837 657 1210 1209 1167 1171 1087 1166 1017 1211 1022 1197 781 1130
## [617] 1216 403 1197 1213 1214 642 1084 607 1205 607 1212 1158 1216 1166
## [631] 1214 1005 919 1132 1199 1186 1196 1175 1134 1173 1176 1093 1074 1203
## [645] 1162 1160 1139 1204 1201 579 1188 1207 474 980 1203 1191 1201 1116
## [659] 1134 1035 1056 1213 621 1189 640 867 1171 1174 646 1194 1129 1063
## [673] 1182 689 1168 1203 1194 836 1053 1196 1146 553 915 561 1214 1209
## [687] 104 122 121 123 123 118 121 123 122 117 122 121 123 118
## [701] 123 122 121 123 122 126 122 122 113 123 120 124 122 121
## [715] 122 124 124 120 122 116 123 118 119 120 123 123 123 120
## [729] 123 123 125 120 122 120 123 120 123 125 121 123 122 122
## [743] 119 123 123 123 121 122 119 124 118 123 122 124 123 115
## [757] 121 120 124 121 122 114 122 123 121 120 121 117 125 119
## [771] 123 119 117 119 123 114 124 120 124 122 121 122 122 123
## [785] 119 124 122 122 116 121 121 122 119 123 114 122 123 122
## [799] 123 122 119 125 123 124 122 123 123 112 125 123 122 121
## [813] 121 123 119 117 123 121 123 119 123 123 120 122 120 123
## [827] 122 121 122 122 120 117 125 121 118 122 122 121 119 121
## [841] 120 123 122 116 119 120 123 123 122 124 122 125 120 125
## [855] 120 117 123 121 125 122 124 123 124 124 119 121 121 123
## [869] 121 122 124 121 124 118 121 121 124 124 124 123 123 122
## [883] 123 113 120 117 124 122 124 123 124 121 118 124 123 123
## [897] 123 122 124 122 123 119 124 123 124 123 122 120 122 120
## [911] 118 122 126 121 123 121 122 122 118 124 120 123 121 124
## [925] 120 124 122 123 123 123 122 118 119 123 121 125 121 122
## [939] 122 126 121 120 122 118 121 122 123 122 125 123 124 122
## [953] 121 124 122 119 124 122 119 126 121 124 120 123 120 123
## [967] 122 120 119 122 122 123 123 124 122 122 124 121 123 118
## [981] 124 119 115 122 123 122 122 120 121 123 120 121 120 125
## [995] 123 120 123 124 124 122 124 125 125 122 123 119 122 121
## [1009] 123 122 120 121 123 120 124 123 122 117 124 122 123 126
## [1023] 124 123 122 126 124 125 124 123 124 117 121 123 122 122
## [1037] 124 124 124 123 123 122 123 123 124 122 125 123 125 124
## [1051] 123 122 124 123 123 121 122 122 123 123 123 122 124 124
## [1065] 121 123 123 115 123 124 120 123 116 125 121 125 123 124
## [1079] 122 122 124 121 123 125 121 124 122 123 121 123 124 124
## [1093] 122 123 124 123 122 120 121 125 123 124 126 123 122 124
## [1107] 123 122 122 120 123 124 123 123 124 123 122 123 122 121
## [1121] 123 122 124 123 124 122 124 123 123 125 125 124 123 122
## [1135] 123 124 112 122 123 123 123 126 123 122 123 123 124 123
## [1149] 123 123 124 125 118 117 124 123 121 123 124 125 124 123
## [1163] 123 124 122 123 123 124 119 126 124 124 122 124 124 116
## [1177] 124 125 122 123 123 118 124 124 123 125 124 124 123 124
## [1191] 122 124 123 121 123 123 125 122 125 123 124 122 125 123

```

```
## [1205] 125 125 123 122 120 122 125 124 122 125 124 122 121 123
## [1219] 119 122 125 124 123 122 124 123 125 125 122 125 125 122
## [1233] 124 125 120 123 124 122 124 123 120 124 124 121 121 120
## [1247] 124 123 124 124 124 123 124 123 126 123 122 120 123 121
## [1261] 124 125 122 122 121 121 123 120 121 124 125 122 125 125
## [1275] 126 124 122 123 123 123 123 124 123 120 124 124 123 122
## [1289] 123 122 121 122 123 124 122 121 123 122 123 124 122 121
## [1303] 124 123 123 123 124 124 124 125 123 120 123 121 123 120
## [1317] 123 123 122 123 121 119 123 121 124 124 124 124 124 125
## [1331] 124 124 121 124 119 123 123 122 123 125 122 123 125 126
## [1345] 125 125 123 126 126 124 125 124 123 120 123 122 121 123
## [1359] 121 124 123 123 124 122 124 123 124 124 124 124 116 123
## [1373] 123 122 123 124 123 124 123 124 122 123 123 119 121 124
## [1387] 122 124 124 122 123 123 124 124 124 122 121 122 124 125
## [1401] 124 122 122 123 123 122 122 124 123 124 125 124 123 122
## [1415] 124 125 124 122 124 124 124 122 123 122 122 124 126 120
## [1429] 125 124 124 123 122 121 120 123 123 125 125 123 124 122
## [1443] 122 124 123 121 120 122 122 124 122 124 123 124 124 120
## [1457] 123 120 121 121 124 124 124 120 124 121 123 125 124 119
## [1471] 121 124 122 116 124 122 125 126 123 124 123 123 123 115
## [1485] 124 122 123 123 122 120 124 124 119 121 121 123 124 124
## [1499] 124 124 123 123 124 124 124 119 122 119 122 124 120 123
## [1513] 124 120 121 104 124 119 124 109 123 124 124 122 123 124
## [1527] 119 124 118 124 122 122 124 123 113 125 125 123 124 123
## [1541] 123 123 121 122 123 114 123 125 124 122 123 124 123 124
## [1555] 124 125 115 112 120 122 123 117 120 124 124 110 121 122
## [1569] 121 123 122 120 123 123 120 123 125 125 119 123 118 116
## [1583] 125 124 113 124 123 123 125 123 124 119 123 119 124 123
## [1597] 116 123 116 124 124 123 123 124 125 123 120 120 122 124
## [1611] 124 124 124 123 123 123 113 123 112 123 122 110 120 122
## [1625] 124 116 123 123 124 119 122 113 109 122 124 120 124 123
## [1639] 124 120 122 122 115 123 120 121 124 125 123 123 124 122
## [1653] 123 123 122 119 122 123 124 125 118 122 116 125 121 123
## [1667] 123 123 118 121 117 117 124 124 122 123 124 123 120
```

```
#Write metadata file
write.csv(smd, "sampleMetadata_combined.csv", quote = FALSE, row.names=FALSE)

#Keep individuals that are not filtered from the dataset
keeplist <- smd$vcfID[which(smd$FILT == FALSE)]

#Write keeplist to a text file
write.table(keeplist, file = "keeperInds.txt",
            quote = FALSE,
            row.names = FALSE,
            col.names = FALSE)
```

Now apply the individual-level filter...

```
vcftools --vcf prefinal.recode.vcf --out finalSCR \
        --keep keeperInds.txt --recode
```



VCFtools - 0.1.17  
(C) Adam Auton and Anthony Marcketta 2009

Parameters as interpreted:

```
--vcf prefinal.recode.vcf
--keep keeperInds.txt
--out finalSCR
--recode
```

```
Warning: Expected at least 2 parts in INFO entry: ID=AF,Number=A,Type=Float,Description="Estimated allele frequency in the sample"
Warning: Expected at least 2 parts in INFO entry: ID=PRO,Number=1,Type=Float,Description="Reference allele ID"
Warning: Expected at least 2 parts in INFO entry: ID=PAO,Number=A,Type=Float,Description="Alternate allele ID"
Warning: Expected at least 2 parts in INFO entry: ID=SRP,Number=1,Type=Float,Description="Strand balance ratio"
Warning: Expected at least 2 parts in INFO entry: ID=SAP,Number=A,Type=Float,Description="Strand balance ratio"
Warning: Expected at least 2 parts in INFO entry: ID=AB,Number=A,Type=Float,Description="Allele balance ratio"
Warning: Expected at least 2 parts in INFO entry: ID=ABP,Number=A,Type=Float,Description="Allele balance ratio"
Warning: Expected at least 2 parts in INFO entry: ID=RPP,Number=A,Type=Float,Description="Read Placement ratio"
Warning: Expected at least 2 parts in INFO entry: ID=RPPR,Number=1,Type=Float,Description="Read Placement ratio"
Warning: Expected at least 2 parts in INFO entry: ID=EPP,Number=A,Type=Float,Description="End Placement ratio"
Warning: Expected at least 2 parts in INFO entry: ID=EPPR,Number=1,Type=Float,Description="End Placement ratio"
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of variant"
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of variant"
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of variant"
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of variant"
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of variant"
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of variant"
Warning: Expected at least 2 parts in INFO entry: ID=CIGAR,Number=A,Type=String,Description="The extended CIGAR string"
Keeping individuals in 'keep' list
After filtering, kept 1679 out of 1851 Individuals
Outputting VCF file...
After filtering, kept 1226 out of a possible 1226 Sites
Run Time = 0.00 seconds
```

## Evaluate Power of the Set

We can use the allele frequencies above to calculate  $p_{ID}$  for the marker set (and for random subsets of the SNP data to give us an idea of how many markers we need to successfully genotype to keep an individual in the dataset).

Here we're using the simple equation from [Waits et al. \(2001\)](#).

For one locus with two alleles:

$$p_{ID} = \sum (p_i^4) + (2p_i p_j)^2$$

Then the power of the combined marker set is the product of  $p_{ID}$  across all SNP loci, as markers are assumed to provide independent information on identity.

First, extract allele frequencies for all loci in the combined dataset.

```
#Extract allele frequencies for these SNPs to calculate p_{ID}
plink2 --vcf finalSCR.recode.vcf --allow-extra-chr --freq --out finalSCR --silent
```

Now use these to calculate  $p_{ID}$ .

```

freqs <- read.table("finalSCR.afreq", header = FALSE)
colnames(freqs) <- c("CHROM", "CPOS", "REF", "ALT", "FRQ", "NCHR")
freqs$pID <- rep(NA, nrow(freqs))
for(l in 1:nrow(freqs)) {
  freqs$pID[l] <- (freqs$FRQ[l]^4+(1-freqs$FRQ[l])^4) +
    (2*freqs$FRQ[l]*(1-freqs$FRQ[l]))^2
}
sum(log10(freqs$pID))

```

```
## [1] -454.2755
```

A more conservative estimate of the power of the set is provided by  $p_{\text{IDSib}}$ , which measures the same probability for siblings. Also from [Waits et al. \(2001\)](#)...

$$p_{\text{IDSib}} = 0.25 + \left(0.5 \sum p_i^2\right) + \left[0.5 \left(\sum p_i^2\right)^2\right] - \left(0.25 \sum p_i^4\right)$$

As above, the  $p_{\text{IDSib}}$  for the dataset is the product across loci.

```

freqs$pIDSib <- rep(NA, nrow(freqs))
for(l in 1:nrow(freqs)) {
  freqs$pIDSib[l] <- 0.25 + (0.5*sum(c(freqs$FRQ[l]^2, (1-freqs$FRQ[l])^2))) +
    (0.5*(sum(c(freqs$FRQ[l]^2, (1-freqs$FRQ[l])^2))^2) -
    (0.25*sum(c(freqs$FRQ[l]^4, (1-freqs$FRQ[l])^4)))
}
sum(log10(freqs$pIDSib))

```

```
## [1] -235.3192
```

## Power with subsets of the panel?

We have exceptional power for individual identification with the full marker set. However, few if any samples will be genotyped at all loci. Here we recalculate the probabilities above with random samples from the full panel (between 10 and 200 SNPs).

```

subsize <- seq(from = 10, to = 200, by = 10)
pIDsub <- matrix(data = NA, nrow = 100, ncol = length(subsize))
pIDSibsub <- matrix(data = NA, nrow = 100, ncol = length(subsize))

for(size in 1:length(subsize)) {
  for(rep in 1:100) {
    tmp.set <- sample(c(1:nrow(freqs)), subsize[size], replace = FALSE)
    tmp.freq <- freqs[tmp.set,]
    tmp.pID <- c()
    tmp.pIDSib <- c()
    for(l in 1:subsize[size]) {
      tmp.pID[l] <- (tmp.freq$FRQ[l]^4+(1-tmp.freq$FRQ[l])^4) +
        (2*tmp.freq$FRQ[l]*(1-tmp.freq$FRQ[l]))^2
      pIDsub[rep,size] <- sum(log10(tmp.pID))
      tmp.pIDSib[l] <- 0.25 +
        (0.5*sum(c(tmp.freq$FRQ[l]^2, (1-tmp.freq$FRQ[l])^2))) +

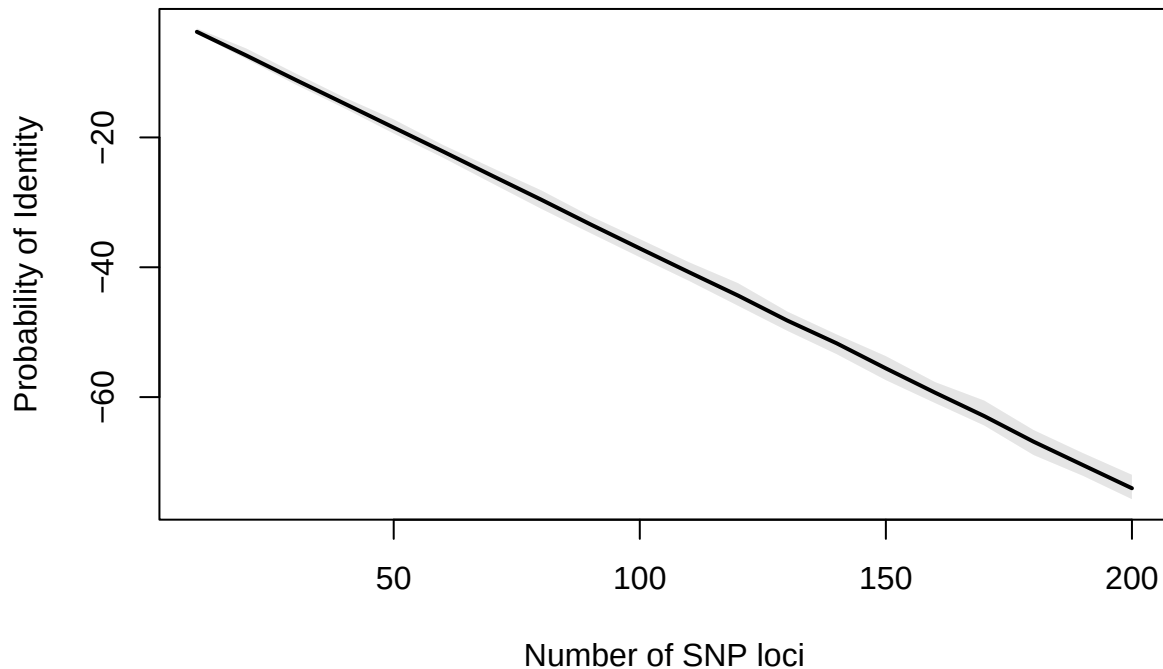
```

```

      (0.5*(sum(c(tmp.freq$FRQ[1]^2, (1-tmp.freq$FRQ[1])^2)))^2) -
      (0.25*sum(c(tmp.freq$FRQ[1]^4, (1-tmp.freq$FRQ[1])^4)))
    pIDSibsub[rep,size] <- sum(log10(tmp.pIDSib))
  }
  rm(tmp.pID, tmp.pIDSib)
  rm(tmp.set, tmp.freq)
}

#Plot the results
library(scales)
plot(subsize, colMeans(pIDSsub), type = "l",
     ylim = range(pIDSsub), lwd = 2, col = "black",
     xlab = "Number of SNP loci", ylab = "Probability of Identity")
quant95 <- apply(pIDSsub, 2, quantile, prob = c(0.025, 0.975))
polygon(x = c(subsize, subsize[length(subsize):1]),
       y=c(quant95[1,], quant95[2,][ncol(quant95):1]),
       col=alpha("black", 0.1), border = NA)

```

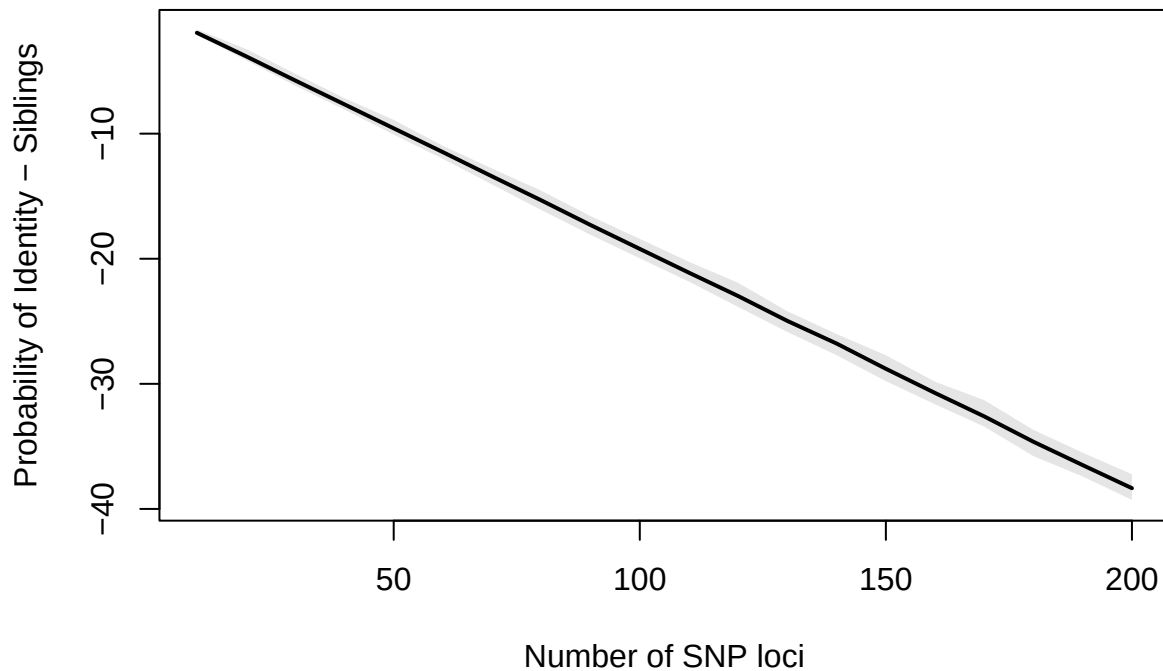


```

plot(subsize, colMeans(pIDSibsub), type = "l",
     ylim = range(pIDSibsub), lwd = 2, col = "black",
     xlab = "Number of SNP loci", ylab = "Probability of Identity - Siblings")
quant95 <- apply(pIDSibsub, 2, quantile, prob = c(0.025, 0.975))
polygon(x = c(subsize, subsize[length(subsize):1]),

```

```
y=c(quant95[1,], quant95[2,][ncol(quant95):1]),
col=alpha("black", 0.1), border = NA)
```



## Test file

As a final test, we'll attempt to use this file with PLINK and vcftools...

```
#Attempt to calculate pairwise kinship coefficients in plink2
plink2 --vcf finalSCR.recode.vcf --make-king-table \
      --out testFinalSCR --allow-extra-chr --silent

#Attempt to use vcftools to make a 012 format file
vcftools --vcf finalSCR.recode.vcf --012 --out testFinalSCR

#Cleanup
rm LGC_allSCRsnps.vcf extraCols.vcf lgc.vcf out.vcf
rm testFinalSCR.log testFinalSCR.kin0
rm *012*
```

VCFTools - 0.1.17

(C) Adam Auton and Anthony Marcketta 2009

Parameters as interpreted:

```
--vcf finalSCR.recode.vcf
--012
--out testFinalSCR
```

```
Warning: Expected at least 2 parts in INFO entry: ID=AF,Number=A,Type=Float,Description="Estimated alle
Warning: Expected at least 2 parts in INFO entry: ID=PRO,Number=1,Type=Float,Description="Reference alle
Warning: Expected at least 2 parts in INFO entry: ID=PAO,Number=A,Type=Float,Description="Alternate alle
Warning: Expected at least 2 parts in INFO entry: ID=SRP,Number=1,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=SAP,Number=A,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=AB,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=ABP,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=RPP,Number=A,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=RPPR,Number=1,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=EPP,Number=A,Type=Float,Description="End Placemen
Warning: Expected at least 2 parts in INFO entry: ID=EPPR,Number=1,Type=Float,Description="End Placemen
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=CIGAR,Number=A,Type=String,Description="The extend
After filtering, kept 1679 out of 1679 Individuals
Writing 012 matrix files ... Done.
After filtering, kept 1226 out of a possible 1226 Sites
Run Time = 0.00 seconds
```

Proceed to recapture analyses in PLINK and COLONY!

---